

Vector Compression for High-Dimensional Data

MATH 5110 · Applied Linear Algebra · github.com/claytomode/math5110-vector-compression

Part 1 — Survey

JL, SVD, sign & scalar quant,
TurboQuant two-stage pipeline

Part 2 — Computation

NumPy implementations, recall@k,
distance distortion metrics

Part 3 — Application

Compressed textbook RAG · 300-
query auto-eval · live search UI

Why Vector Compression Matters

Modern AI stores **billions of high-dimensional vectors** — embeddings, KV-cache keys, retrieval indexes.

Storage scales as $n \cdot d \cdot \text{bits/dim}$.

RAG Indexes

Document-chunk embeddings ·
index size and query bandwidth
scale with corpus size

KV Cache

Attention key vectors per token ·
memory grows with context
length

Embedding Tables

One vector per token type · table
size grows with vocabulary and
dimension

❗ Naive float32 storage: $32 \cdot n \cdot d$ bits. Can we cut bits per dimension without breaking geometry?

Four Compression Strategies

We investigated four compression strategies:

→ **JL Random Projection**

Sketch $y=Rx$, $k=O(\log n/\epsilon^2)$

→ **Rank-k SVD**

Truncate spectral subspace via polar geometry

→ **Sign 1-bit Quantization**

Store $\text{sign}(x)$, use QJL-style scoring

→ **Scalar Uniform Quantization**

b bits per coordinate for direct control

Each method trades compression ratio against geometry preservation — distance error, inner-product accuracy, and top-k retrieval ranking are not equivalent objectives.

JL Random Projection

What It Does

Multiply each vector by a random Gaussian matrix R to project from d dimensions down to k . Random matrices are near-isometries — they approximately preserve all pairwise distances.

The Map

$$\square \quad y = Rx, \quad R \in \mathbb{R}^{k \times d}, \quad R_{ij} \sim N(0, 1/k)$$

The JL Guarantee

For any n points and error tolerance ϵ , a target dimension k exists such that all pairwise distances are preserved within factor $(1 \pm \epsilon)$:

$$\square \quad k = O(\epsilon^{-2} \log n)$$

Our Setting

$$\square \quad d=256, \epsilon=0.3, n=1380 \rightarrow k \approx 128 \rightarrow 2\times \text{compression}$$

Key Insight

⚠ Distance preservation $\neq$ rank preservation. JL guarantees geometry, not retrieval order.

RAG Result

$\square$ Overlap@3 = 39.2% — worst performer despite theoretical guarantees

Rank-k SVD (Truncated)

What It Does

Factor the full embedding matrix into three matrices U, Σ, V^T . Keep only the top-k singular directions — the axes that capture the most variance in the data.

The Decomposition

$$\square \quad A = U\Sigma V^T \rightarrow A_k = U_k \Sigma_k V_k^T$$

Compress & Reconstruct

Project any vector into the k-dimensional spectral subspace. The Eckart–Young theorem guarantees this is the best possible rank-k approximation:

$$\square \quad z = V_k^T x \in \mathbb{R}^k \rightarrow \hat{x} = V_k V_k^T x$$

Our Setting

$$\square \quad d=256, k=64 \rightarrow 4\times \text{compression}$$

Key Insight

- ✓ Theoretically optimal linear compressor — no rank-k method beats SVD in Frobenius norm.

Limitation

- Must fit V_k on full corpus upfront. Out-of-distribution vectors lose alignment.

Sign 1-bit Quantization

What It Does

Replace every coordinate with just its sign — +1 or -1. The most aggressive compression possible: 32 bits per coordinate becomes 1 bit.

The Encoding

$$\hat{x} = \text{sign}(x) \in \{-1, +1\}^d$$

QJL Scoring

Apply a random rotation R before quantizing. The angle between rotated sign vectors encodes the original cosine similarity:

$$\text{score}(q, \hat{x}) = \text{sign}(Rx)^T(Rq) / \sqrt{d} \approx \cos(x, q)$$

Compression

$$1 \text{ bit/dim} \rightarrow 32\times \text{ vs float32}$$

Key Insight

⚠ Works when vectors are well-separated. Fails when top-k cluster tightly.

RAG Result

$$\text{Overlap@3} = 47.9\% \text{ at } 28\times \text{ compression}$$

Scalar Uniform Quantization

What It Does

Divide the coordinate range into 2^b equal buckets and round each value to the nearest bucket center.

The Step Size

$$\Delta = (x_{\max} - x_{\min}) / (2^b - 1)$$

Error Bound

Each coordinate rounds by at most $\Delta/2$. Each extra bit halves Δ — a clean, predictable quality-vs-bits tradeoff:

$$\|x - \hat{x}\|^2 \leq d \cdot (\Delta/2)^2$$

Bits → Compression

$$b=8 \rightarrow 4\times \quad b=4 \rightarrow 8\times \quad b=2 \rightarrow 16\times$$

Key Insight

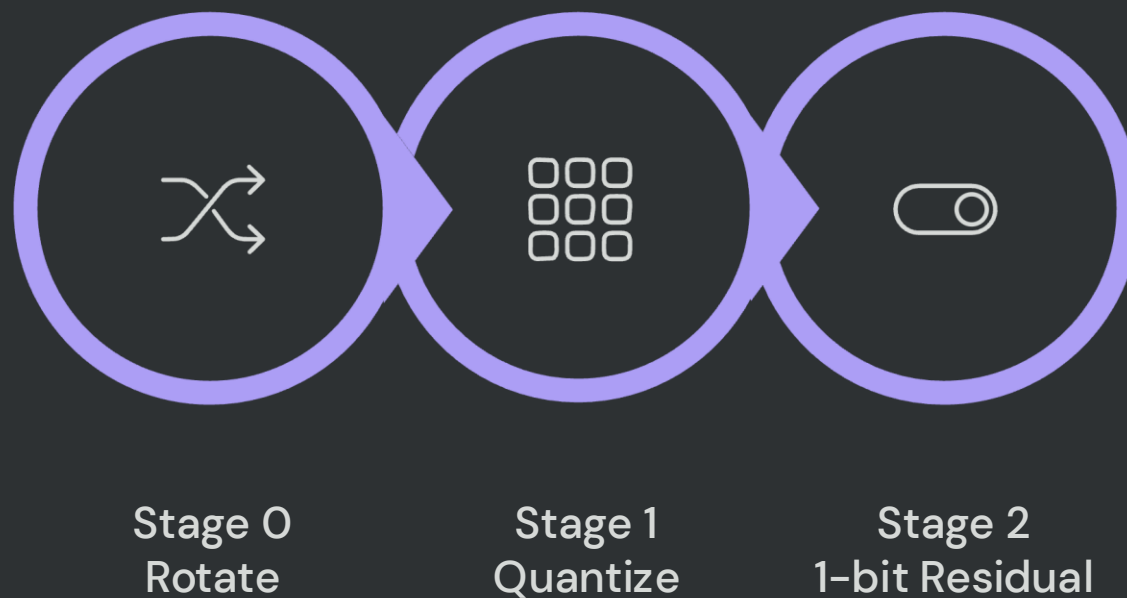
✓ At 4–8 bits it matches or beats more complex methods.

RAG Results

SC-8: 99.7% · SC-4: 93.2% · SC-2: 62.8% Overlap@3

TurboQuant: Two-Stage Pipeline

From [Google Research \(ICLR 2026\)](#) — implemented here in pedagogical form on course embeddings.



✓ **Scoring formula:** $\hat{q}^T x \approx \hat{q}^T \hat{x}_1 + \|r\| \cdot \text{sign}(r)^T q / \sqrt{d}$ — full-precision query corrects the stage-1 bias at zero storage cost.

Simplified TurboQuant: JL + Scalar Quantization

Stage 1 — Scalar Quantize

$$\hat{x}_1 = \text{scalar_quantize}(x, b \text{ bits})$$

Stage 2 — Residual Correction

$$r = x - \hat{x}_1 \rightarrow \hat{r} = \text{sign}(Rr)$$

Scoring

$$\text{score}(q, x) \approx q^T \hat{x}_1 + \|r\| \text{sign}(Rr)^T (Rq) / \sqrt{d}$$

Why It Works

- ✓ The residual r is small. JL handles the correction cheaply.

True Bit Budget

$$\text{TQ-2} = \sim 3.2 \text{ bits/dim (not 2.0)}$$

Recall Gain

- ✓ sign_1bit: 47.9% → TQ-2: 76.1% Overlap@3.

Experimental Setup

Corpus

- ❏ Prof. Wang's MATH 5110 textbook · 1,380 chunks · 512 tokens/chunk · 50-token overlap

Query Set

- ❏ 300 stratified queries · auto-generated · evaluated vs full-precision ground truth

Embedding Model

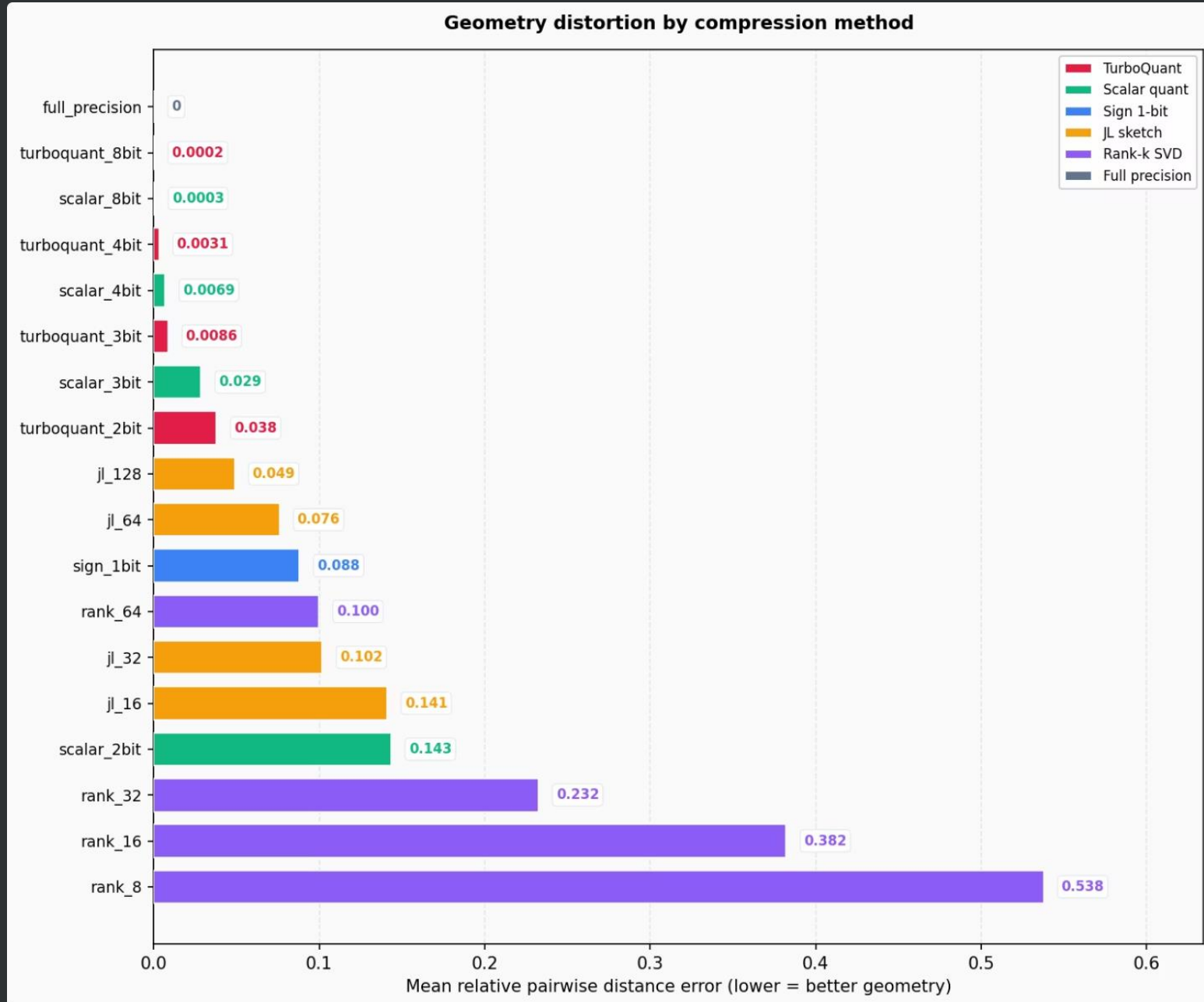
- ✅ text-embedding-3-small · $d = 256$ · L2-normalized · cosine = dot product

Evaluation Metric

- ❏ Overlap@k — fraction of top-k chunks matching full-precision top-k · baseline = 100% by definition

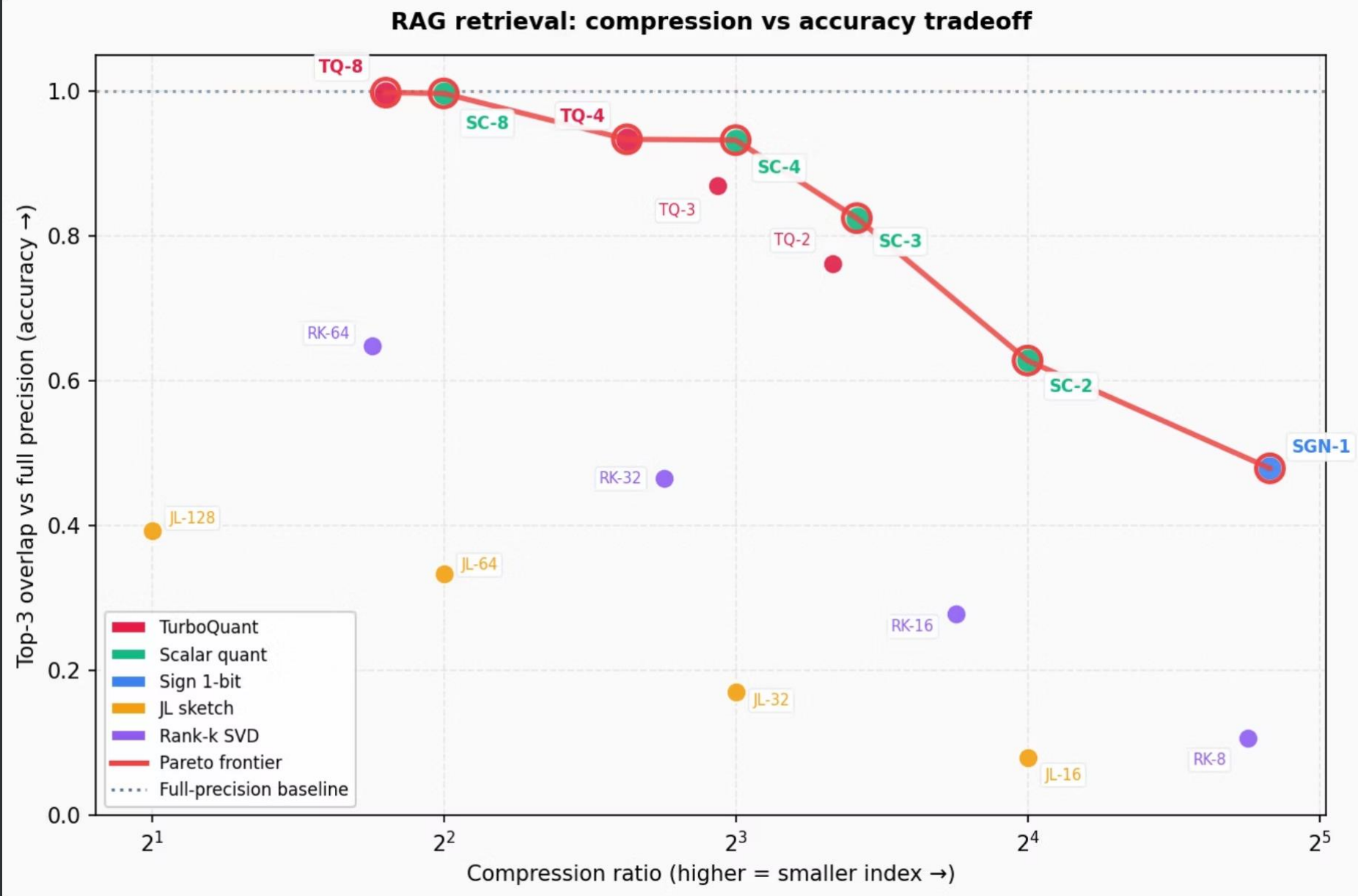
Geometric Distortion

RESULTS



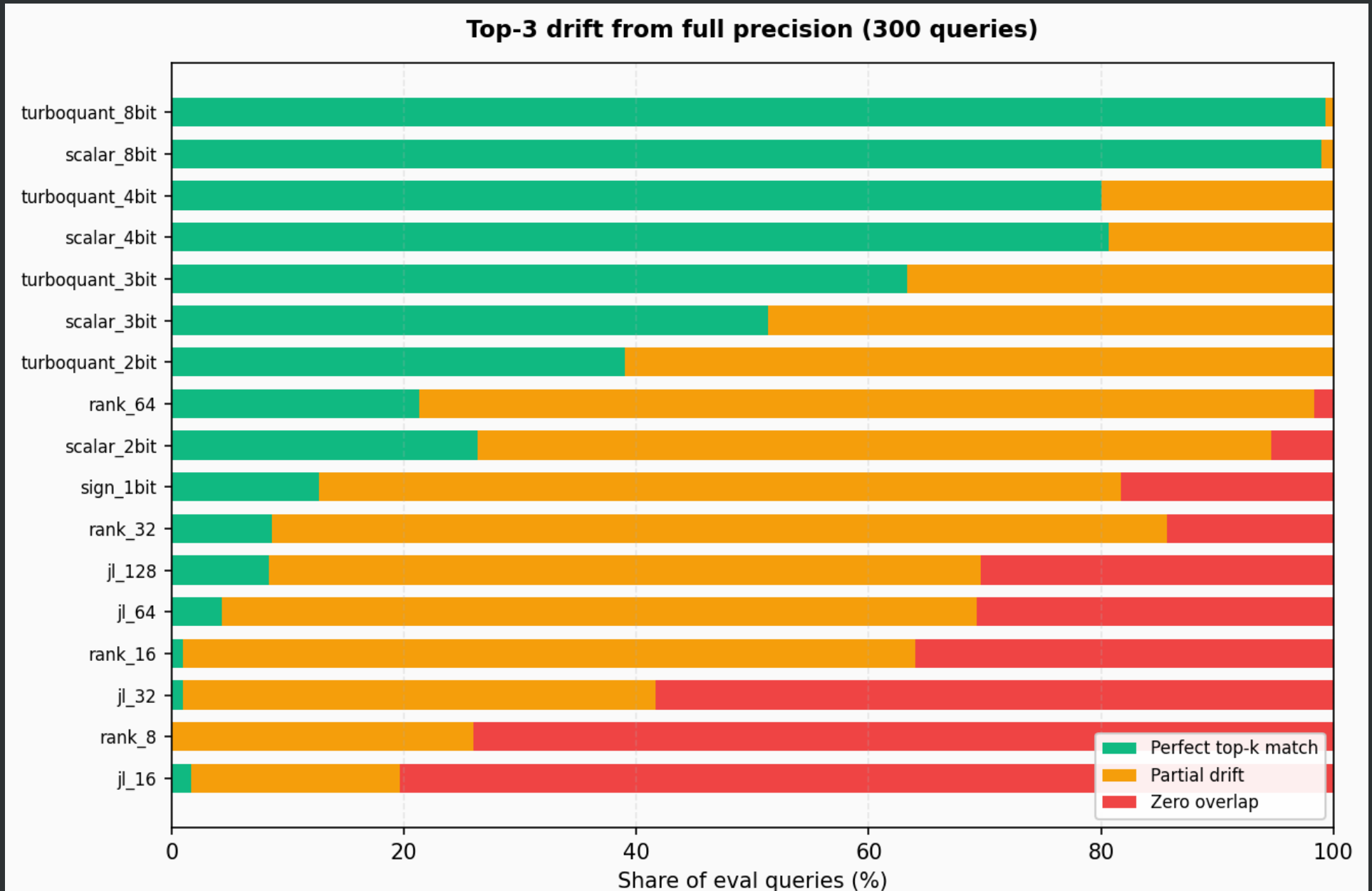
RAG Compression Frontier

RESULTS



RAG top-k drift from Float32

RESULTS



Takeaways

TurboQuant wins at aggressive compression

When stage-1 bits are already tight (2–3 bits), the JL residual correction earns its overhead — TQ-2 beats SC-2 by 13 points (76% vs 63% Overlap@3).

But scalar wins when you have room

At 4–8 bits, scalar quantization matches TurboQuant on recall while using fewer total bits. Simpler, faster, no rotation overhead.

JL distance theory doesn't imply retrieval

jl_128 has bounded distance distortion but only 39% RAG overlap. Geometry preservation $\neq$ rank preservation.

Labels are not total bit budgets

TQ-2 costs ~3.2 bits/dim, not 2.0. Always compare methods on actual bits/dim, not stage-1 labels.

Search

Question

What is the spectral theorem for symmetric matrices?

What is the spectral theorem for symmetric matrices?

What is the singular value decomposition?

Explain the Perron-Frobenius theorem

Index type

Top K

Full float32



5



☒ Compare indexes

Search

Ctrl+Enter to search

Results

Expand all

Compared 11 indexes · 1,380 chunks · d=256

Full float32

full_precision

- #1 Ch. 13 — Symmet... 0.741
- #2 Ch. 13 — Symmet... 0.730
- #3 Ch. 13 — Symmet... 0.698
- #4 Ch. 07 — Diagon... 0.668
- #5 Ch. 14 — Singular... 0.661

TurboQuant 2-bit

turboquant_2bit

- #1 Ch. 13 — Symmet... 0.735
- #2 Ch. 13 — Symmet... 0.727
- #3 Ch. 07 — Diagon... 0.679
- #4 Ch. 14 — Singular... 0.667
- #5 Ch. 13 — Symmet... 0.664

TurboQuant 3-bit

turboquant_3bit

- #1 Ch. 13 — Symmet... 0.735
- #2 Ch. 13 — Symmet... 0.735
- #3 Ch. 13 — Symmet... 0.698
- #4 Ch. 07 — Diagon... 0.672
- #5 Ch. 14 — Singular... 0.666

TurboQuant 4-bit

turboquant_4bit

- #1 Ch. 13 — Symmet... 0.743
- #2 Ch. 13 — Symmet... 0.725
- #3 Ch. 13 — Symmet... 0.696
- #4 Ch. 07 — Diagon... 0.673
- #5 Ch. 14 — Singular... 0.661

"Demo"